上讲主要内容:

- DH交换协议
- ElGamal密码
- 椭圆曲线密码
- IBE算法

《密码学基础》

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

第11讲 哈希函数

消息完整性



授课教师：徐国爱

C内容 ontents



1 概述

2 HASH算法

3 HASH安全性

4 MAC

1.1 哈希（Hash）函数简介

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

- 别名：散列函数、杂凑函数、数字摘要、数字指纹、消息摘要、哈希值或散列值等
- 方案：一种单向密码体制，即它是一个从明文到密文的不可逆映射，即只有“加密”过程，不存在“解密”过程。同时，Hash函数可以将“任意”长度的输入经过变换以后得到固定长度的输出
- 应用：哈希函数在数据完整性和数字签名等领域有广泛的应用，在现代密码学中起着非常重要作用

1.1 哈希（Hash）函数简介-公式表示

对不同长度的输入消息，产生固定长度的输出。这个固定长度的输出称为原输入消息的“哈希值”或“消息摘要”（Message Digest）。

公式表示形式：

$$h=H(M)$$

M：*任意*长度的消息

H：哈希（Hash）函数或杂凑函数或散列函数

h：固定长度的哈希值

1.2 哈希算法的性质(特点)

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

- ✓ 输入：消息是*任意有限*长度。
- ✓ 输出：哈希值是固定长度。
- ✓ 容易计算：对于任意给定的消息，容易计算其哈希值。（正向容易）
- ✓ 单向性：对于给定的哈希值h，要找到M使得H(M)＝h在计算上是不可行的。

  （逆向不可行）

1.2 哈希算法的性质(安全性)

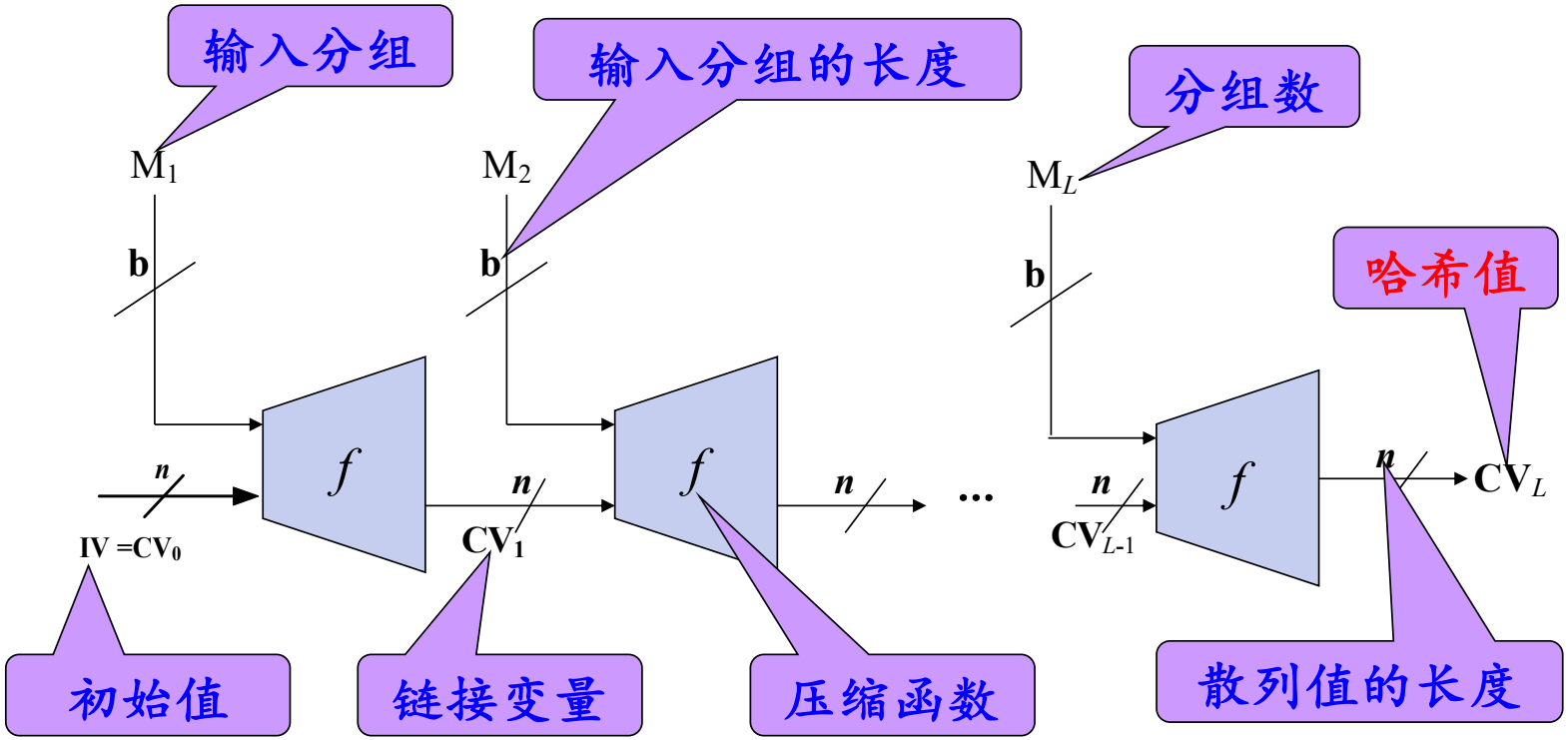
哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

注：碰撞是指不同的消息M_1和消息M_2使得$H(M_1)=H(M_2)$

- ✓ 抗弱碰撞性：对于给定的消息M_1，要发现另一个消息M_2，满足$H(M_1)=H(M_2)$在计算上是不可行的。
- ✓ 抗强碰撞性：找任意一对不同的消息M_1, M_2，使$H(M_1)=H(M_2)$在计算上是不可行的。
- ✓ 随机性：当一个输入位发生变化时，输出位将发生很大变化。(雪崩效应)

1.3 哈希函数设计（一般结构）

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN



1.3 哈希函数设计（核心技术）

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

设计无碰撞的Hash函数f，而攻击者对算法的攻击重点是Hash函数f 的内部结构，由于Hash函数f 和分组密码一样是由若干轮处理过程组成，所以，对Hash函数f 的攻击需通过对各轮之间位模式的分析来进行，分析过程常常需要先找出Hash函数f 的碰撞。由于Hash函数f是压缩函数，其碰撞是不可避免的。因此，在设计Hash函数f时就应保证找出其碰撞在计算上是不可行的。

常用的散列函数：

MD5；

SHA系列；

1.4 哈希函数历史

- Hash的概念起源于1956年，Dumey用它来解决symbol table question(符号表问题)，使得数据表的插入、删除、查询操作可以在更短的时间内完成。
- 散列算法MD族是在上个世纪90年代初由Ron·Rivest设计的，MD代表消息摘要(message-digest)， MD2(1989)、MD4(1990)和MD5(1991)都产生一个128位的信息摘要。
- SHA系列算法是美国国家标准与技术研究院(NIST)根据Rivest 设计的MD4和MD5而开发的算法，国家安全当局发布SHA作为美国政府标准，SHA(Secure Hash Algorithm)表示安全散列算法。

Contents 内容



1 概述

2 HASH算法

3 HASH安全性

4 MAC

2.1 SHA系列简介

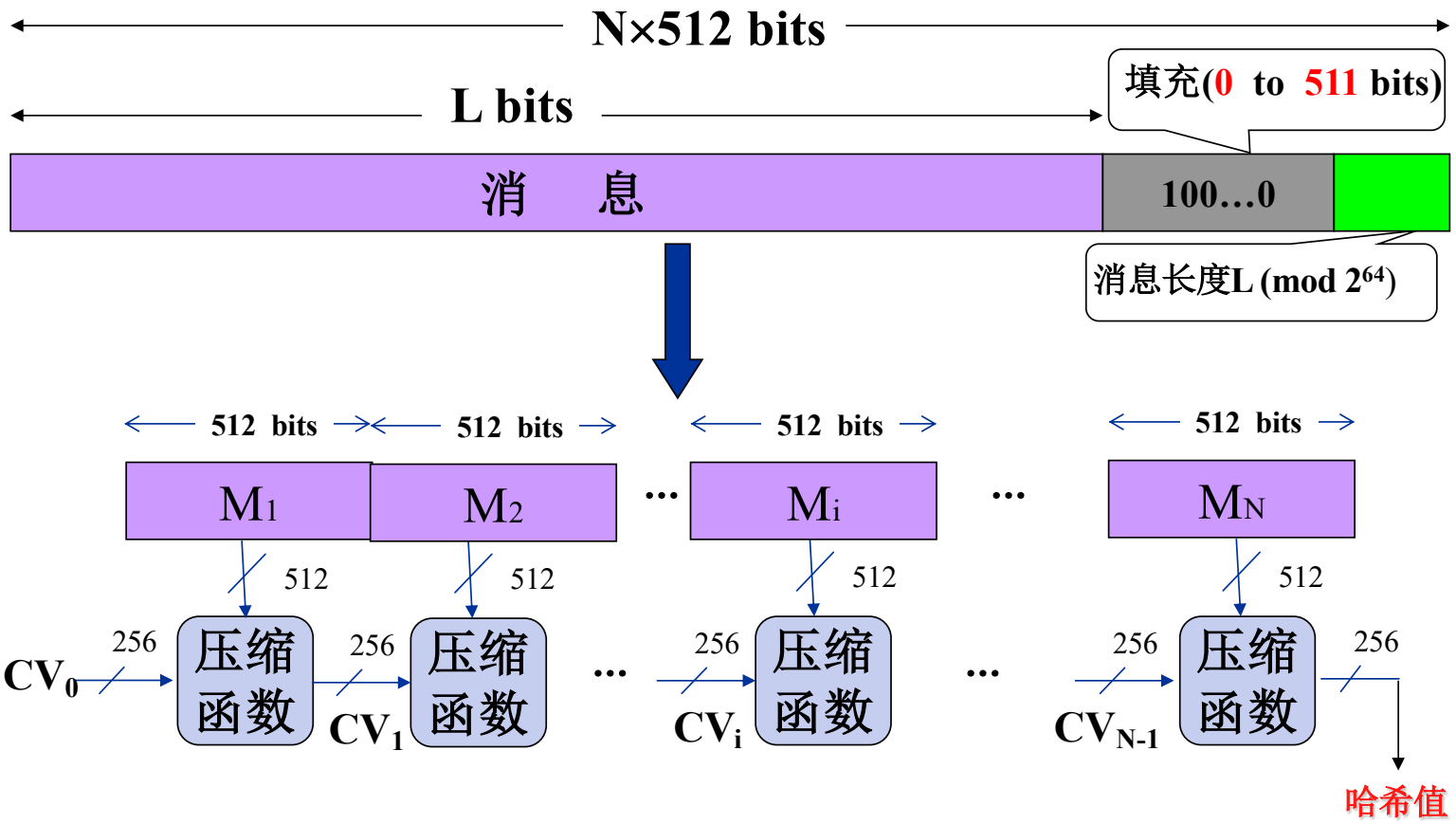
SHA系列包括多个散列算法标准，其中，SHA-1是数字签名标准中要求使用的算法。

- SHA-0：正式地称作SHA，这个版本在发行后不久被指出存在弱点。
- SHA-1：NIST于1994年发布的，它与MD4和MD5散列算法非常相似，被认为是MD4和 MD5的后继者。(160位)
- SHA-2：实际上分为SHA224、SHA256、SHA384和SHA512 、SHA512/224 、SHA512/256算法。

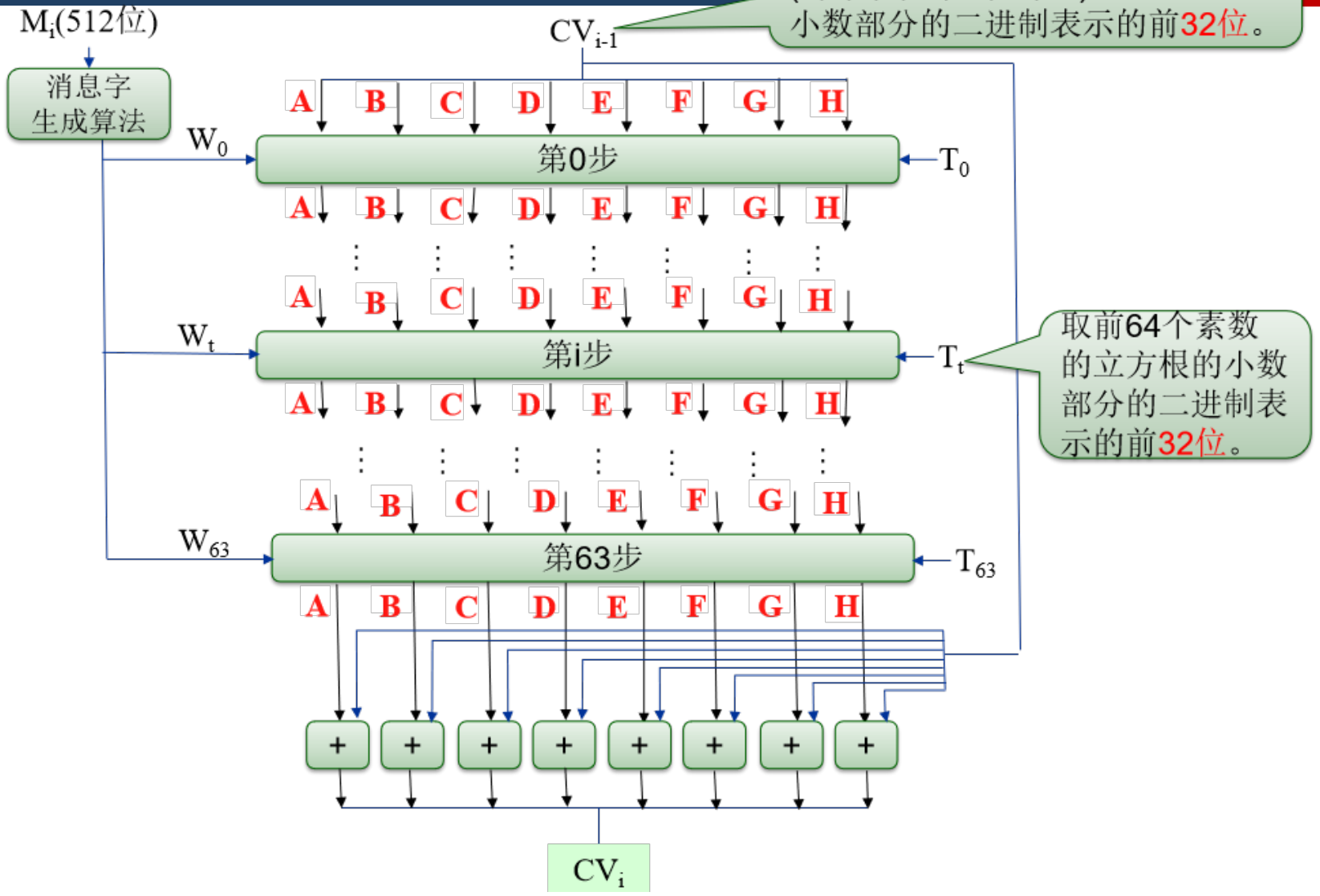
2.1 SHA系列简介-SHA256

SHA256接受任何有限长度的输入消息，并产生长度为256比特的Hash值(MD5和SHA-1分别生成128位和160位的哈希值)，因此，抗穷举攻击的能力更强。SHA-256消息分组和填充方式与SHA-1相同，其设计原理与SHA-1类似，但SHA-256有8个参与运算的32位寄存器字，主循环包括64步循环模块，每步函数包括非线性运算、循环移位、异或运算操作和加法运算操作等。

2.1 SHA256简介-哈希值的生成过程

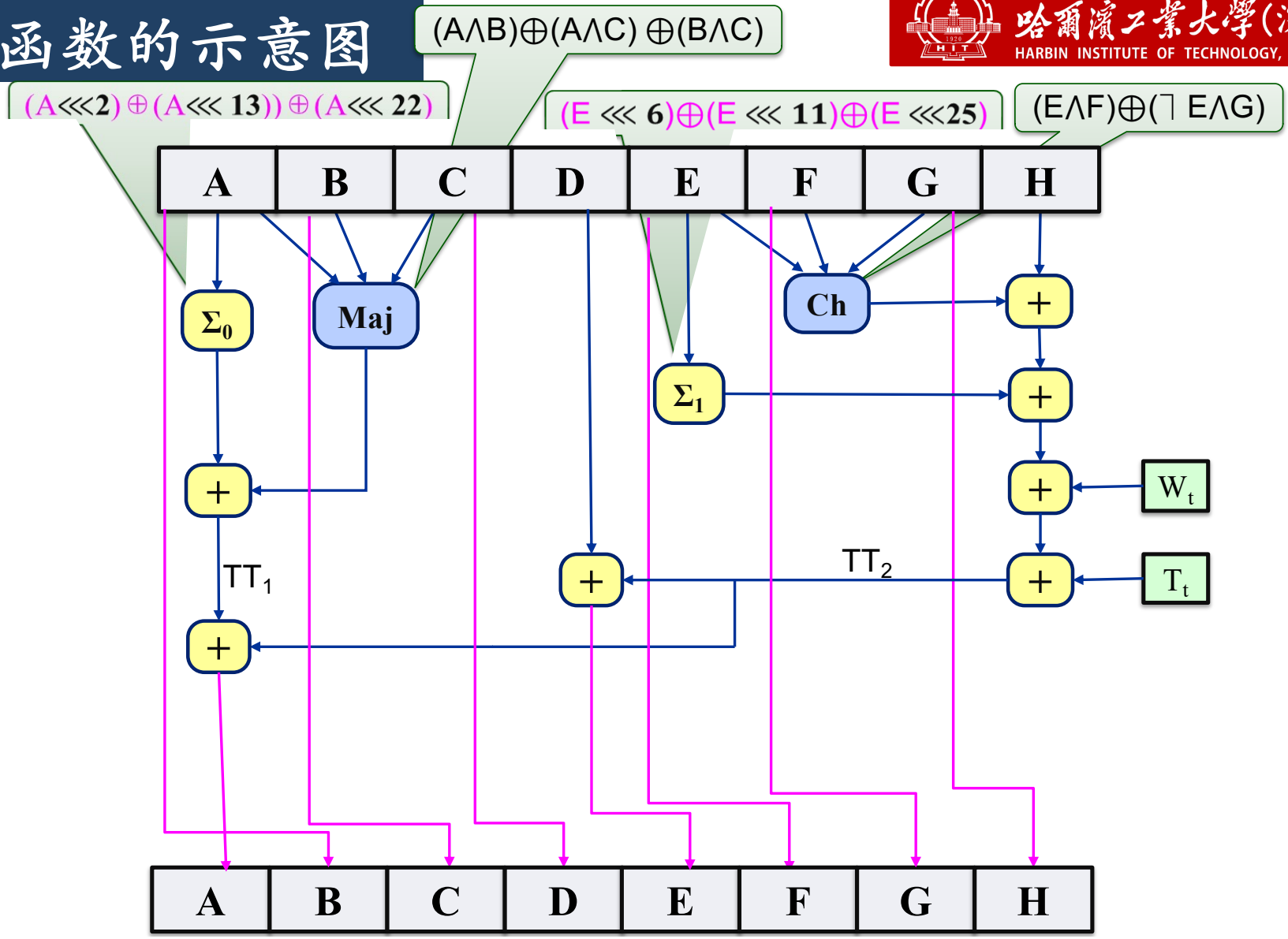


2.2 SHA单个512位分组的处理过程



2.2 步函数的示意图

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN



2.2 步函数的计算过程

$(A \lll 2) \oplus (A \lll 13)) \oplus (A \lll 22)$

$(E \lll 6) \oplus (E \lll 11) \oplus (E \lll 25)$

$(A \wedge B) \oplus (A \wedge C) \oplus (B \wedge C)$

$(E \wedge F) \oplus (\neg E \wedge G)$

$T_1 = \Sigma_0(A) + Maj(A,B,C) \bmod 2^{32}$

$T_2 = \Sigma_1(E) + Ch(E,F,G) + H + W_t + T_t \bmod 2^{32}$

$A = (TT_1 + TT_2) \bmod 2^{32}$

$B = A$

$C = B$

$D = C$

$E = (D + TT_2) \bmod 2^{32}$

$F = E$

$G = F$

$H = G$

SHA256散列函数中涉及的操作全部是逻辑的位运算

包括如下的逻辑函数：

$Ch(x,y,z)=(x\wedge y)\oplus(\neg x\wedge z)$

$Ma(x,y,z)=(x\wedge y)\oplus(x\wedge z)\oplus(y\wedge z)$

$\Sigma 0(x)=S2(x)\oplus S13(x)\oplus S22(x)$

$\Sigma 1(x)=S6(x)\oplus S11(x)\oplus S25(x)$

$\sigma 0(x)=S7(x)\oplus S18(x)\oplus R3(x)$

$\sigma 1(x)=S17(x)\oplus S19(x)\oplus R10(x)$

逻辑运算	含义
$\wedge$	按位“与”
$\neg$	按位“补”
$\oplus$	按位“异或”
S^n	循环右移n个bit
R^n	右移n个bit

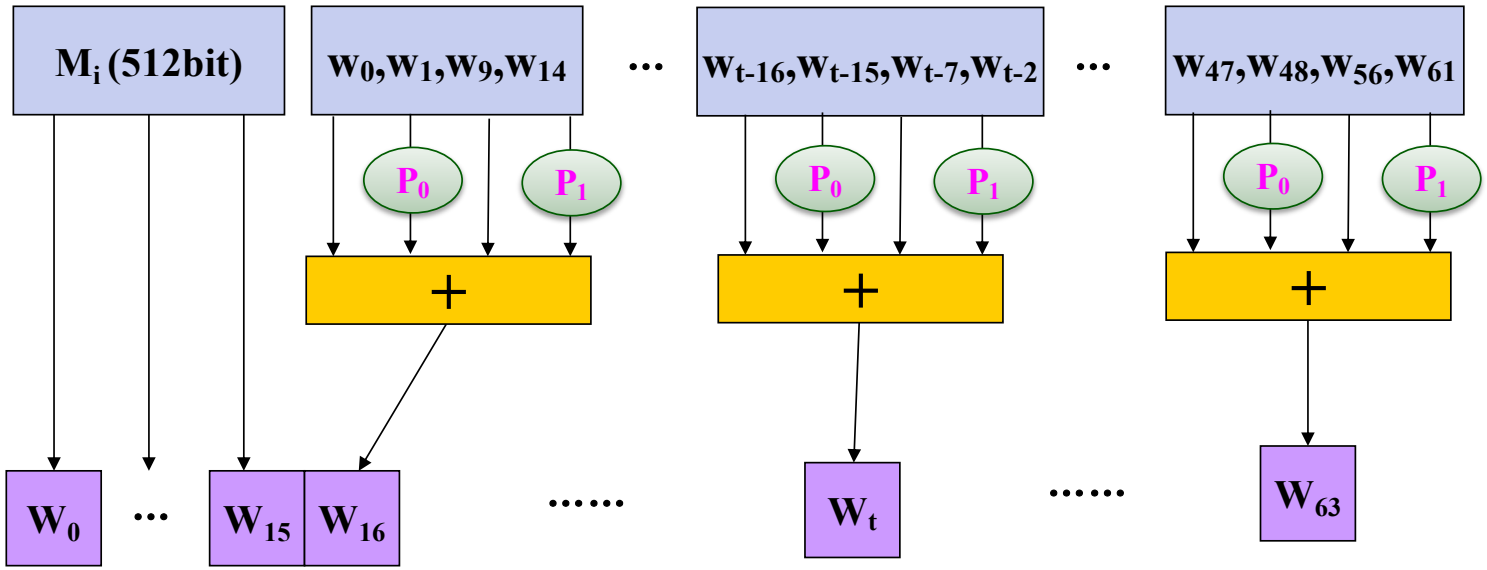
https://blog.csdn.net/leho666

2.2 单个512位分组生成64个字的方法

$(X \lll 7) \oplus (X \lll 18) \oplus (X \ll 3)$

$(X \lll 17) \oplus (X \lll 19) \oplus (X \ll 10)$

$$W_t = W_{t-16} + P_0(W_{t-15}) + W_{t-7} + P_1(W_{t-2}) \quad (16 \le t \le 63)$$



2.3 SM3简介

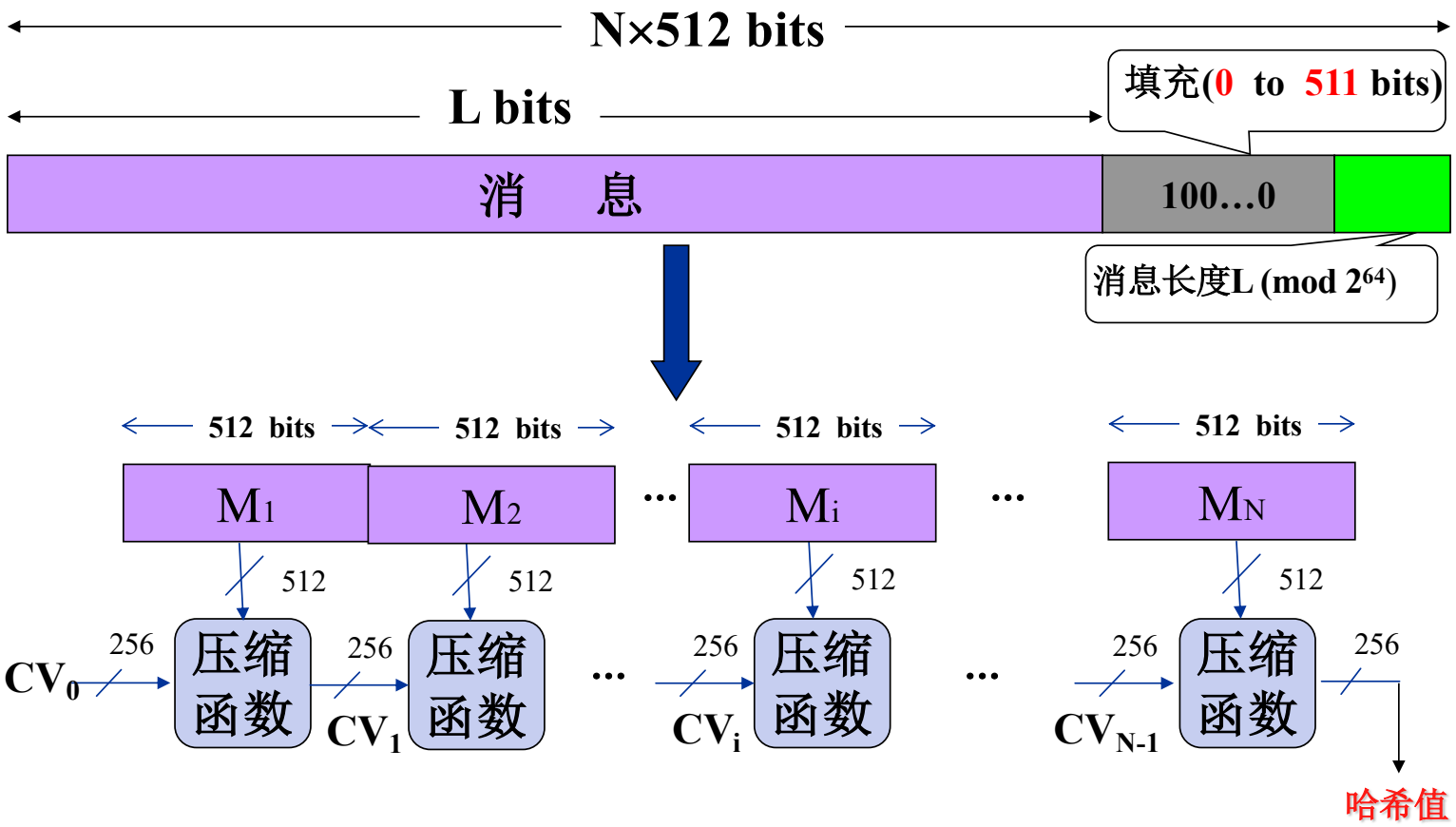
SM3是我国采用的一种密码散列函数标准，由国家密码管理局于2010年12月17日发布，其主要用于数字签名及验证、消息认证码生成及验证、随机数生成等，其算法公开，据国家密码管理局表示，其安全性及效率与SHA-256相当。

对长度为L(< 2^{64}) 比特的消息m， SM3杂凑算法经过填充和迭代压缩，生成杂凑值，杂凑值长度为256比特。

SM3算法的消息分组长度为512比特，如果不满足512比特分组，需要填充，其填充方法同SHA-256填充方法完全一致。

2.3 SM3算法的流程图

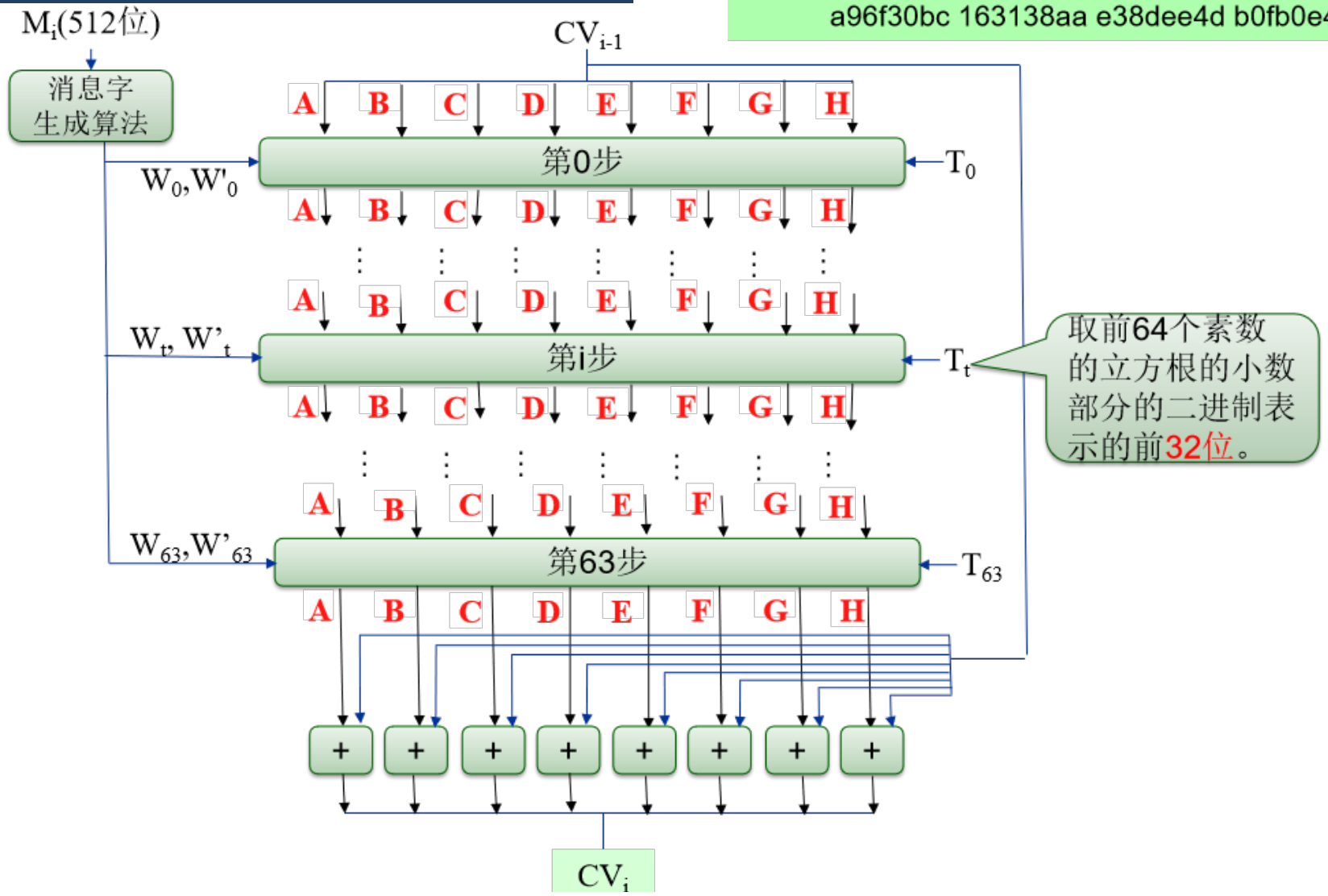
哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN



CV_0= 7380166f 4914b2b9 172442d7 da8a0600 a96f30bc 163138aa e38dee4d b0fb0e4e

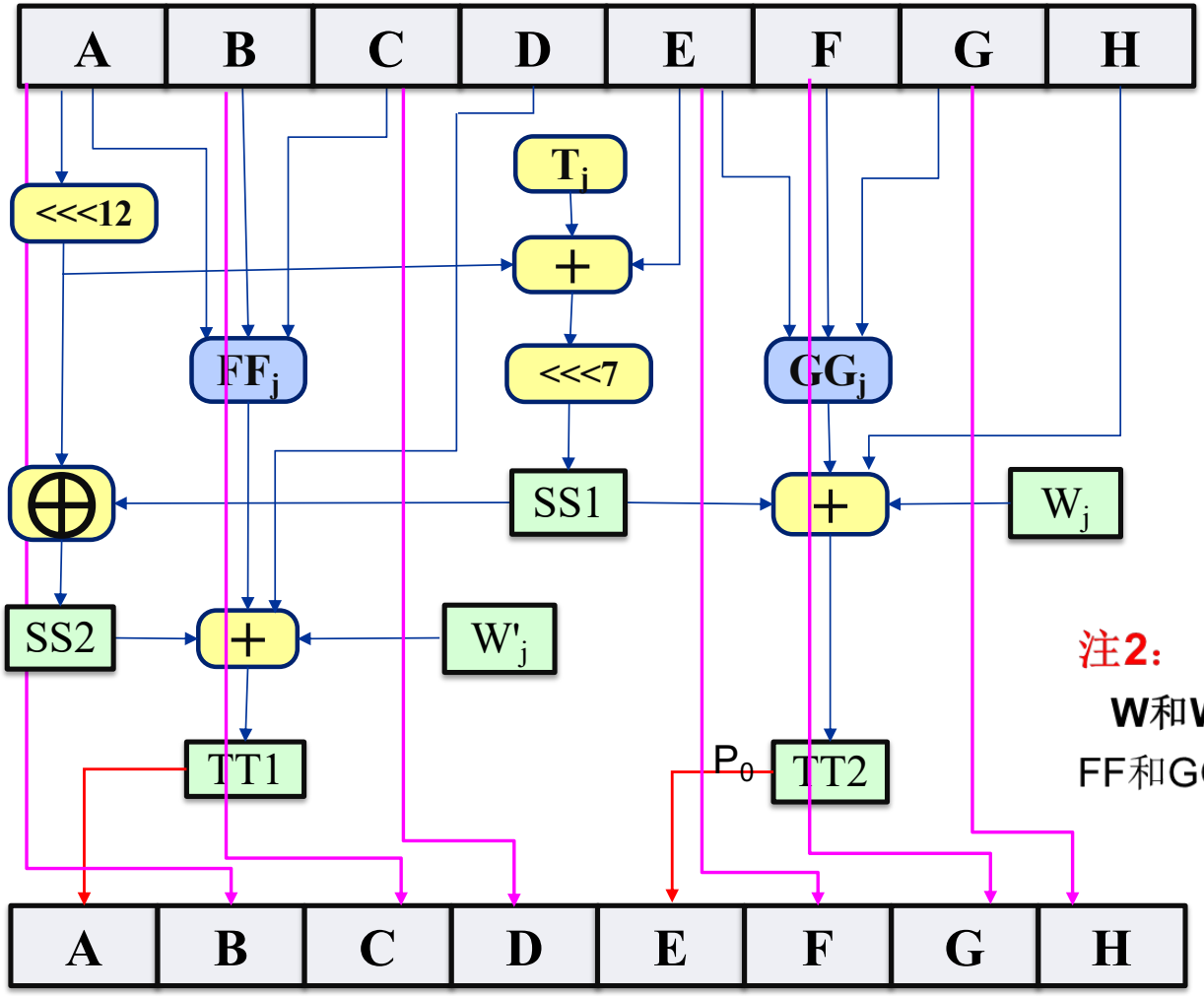
2.4 单个512位分组的处理过程

CV_0 = 7380166f 4914b2b9 172442d7 da8a0600
a96f30bc 163138aa e38dee4d b0fb0e4e



2.4 压缩函数的示意图

注1：A,B,C,D,E,F,G,H为字寄存器
SS1,SS2,TT1,TT2为中间变量



注2：

W和W′:与M_i有关

FF和GG :逻辑函数

T:固定常数

2.4 压缩函数计算过程

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

注：A,B,C,D,E,F,G,H的初始值来自IV，即CV_0=IV

CV_0 = 7380166f 4914b2b9 172442d7 da8a0600 a96f30bc 163138aa e38dee4d b0fb0e4e

$ABCDEFGH \leftarrow V^{(i)}$

FOR j=0 TO 63

$SS1 \leftarrow ((A \lll 12) + E + (T_j \lll j)) \lll 7$

$SS2 \leftarrow SS1 \oplus (A \lll 12)$

$TT1 \leftarrow FF_j(A,B,C) + D + SS2 + W'_j$

$TT2 \leftarrow GG_j(E,F,G) + H + SS1 + W_j$

$D \leftarrow C$

$C \leftarrow B \lll 9$

$B \leftarrow A$

$A \leftarrow TT1$

$H \leftarrow G$

$G \leftarrow F \lll 19$

$F \leftarrow E$

$E \leftarrow P_0(TT2)$

ENDFOR

$V^{(i+1)} \leftarrow ABCDEFGH \oplus V^{(i)}$

$$T_j = \begin{cases} 79cc4519, & 0 \le j \le 15 \\ 7a879d8a, & 16 \le j \le 63 \end{cases}$$

$$FF_j(X,Y,Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \le j \le 15 \\ (X \wedge Y) \vee (X \wedge Y) \vee (Y \wedge Z), & 16 \le j \le 63 \end{cases}$$

$$GG_j(X,Y,Z) = \begin{cases} X \oplus Y \oplus Z, & 0 \le j \le 15 \\ (X \wedge Y) \vee (\neg X \wedge Z), & 16 \le j \le 63 \end{cases}$$

$$P_0(X) = X \oplus (X \lll 9) \oplus (X \lll 17)$$

2.4 分组消息的扩展

分组消息扩展是根据分组消息(512比特)扩展成132个字：

$$W_0, W_1, \cdots, W_{67}, W'_0, W'_1, \cdots, W'_{63}$$

其过程如下：

a)将分组消息划分为16个字 $W_0, W_1, \cdots, W_{15}$。

b)FOR j=16 TO 67

$$W_j \leftarrow P_1(W_{j-16} \oplus W_{j-9} \oplus (W_{j-3} \lll 15)) \oplus (W_{j-13} \lll 7) \oplus W_{j-6}$$

ENDFOR

c)FOR j=0 TO 63

$$W'_j = W_j \oplus W_{j+4}$$

ENDFOR

$$P_1(X) = X \oplus (X <<< 15) \oplus (X <<< 23)$$

2.5 SM3的评价

SM3密码杂凑算法压缩函数整体结构与SHA-256相似，但是增加了多种新的设计技术，包括增加16步全异或操作、消息双字介入、增加快速雪崩效应的P置换等。能够有效地避免高概率的局部碰撞，有效地抵抗强碰撞性的差分分析、弱碰撞性的线性分析和比特追踪法等密码分析。

SM3密码杂凑算法合理使用字加运算，在不显著增加硬件开销的情况下，采用P置换，加速了算法的雪崩效应，提高了运算效率。同时，SM3密码杂凑算法采用了适合32位微处理器和8位智能卡实现的基本运算，具有跨平台实现的高效性和广泛的适用性。

Contents 内容



1 概述

2 HASH算法

3 HASH安全性

4 MAC

3.1 哈希函数的攻击

- 攻击者的主要目标不是恢复原始的明文，而是寻找与合法消息哈希值相同的非法消息，然后用非法消息替代合法消息进行伪造和欺骗，所以，对哈希函数的攻击也是寻找碰撞的过程。

- 暴力攻击
- 生日攻击

3.2 暴力攻击

指在不改变文档内意思的前提下能够对文件的内容进行修改的程度。

主动攻击者Mallory得到Alice已签署的合同文件，Mallory欲将合同中“Alice要支付的金额为100万元”改为“Alice要支付的金额为1亿元”

Mallory 可以从文档文件所具有的冗余性入手。

譬如，下面这些句子基本上都是一个意思。

Alice要支付的金额为1亿元。
Alice要支付的金额为壹亿元。
Alice要支付的金额为100000000元。
Alice要支付的金额为￥100000000。
Alice要支付的金额为:1亿元。
Alice需要支付的金额为1亿元。
Alice应支付1亿元。
作为报酬，Alice需要支付1亿元。

暴力攻击就是利用文件的冗余性生成具有相同哈希值且对攻击者有利的另一文件。暴力攻击是一种试图破解哈希函数的“抗弱碰撞性”的攻击。

3.2 暴力攻击-弱碰撞和强碰撞的举例

举个例子，想象一位老师问一个有30名学生的班级(n=30)每个人的生日在哪一天(为简便，此处省略闰年)以确定是否有两个学生同一天生日(对应碰撞)。从直觉角度考虑，机率看起来很小。若老师选择特定日期(例如9月16日)，则至少有一名学生在那天出生的几率是1-(364/365)，约为7.9%(弱碰撞概率)。但是，与我们的直觉相反的是，至少一名学生和另外任意一名学生有着相同生日的几率大约为70.63%(强碰撞概率)(n=30时)，从$1-365!/((365-n)!\cdot 365^n)$可看出。(生日悖论)

$$\frac{365.364\cdots(365-(n-1))}{365.365\cdots 365}$$

3.2 暴力攻击-生日攻击结论

- ✓ 问题:假定每个人的生日是等概率的，每年有365天(不考虑闰年)。在k个人中至少有两个人的生日相同的概率大于0.5，问k的最小值是多少?
- ✓ 把每个人的生日看作在[1，365]中的随机变量。由组合数学基本知识得知k个人的生日出现重复的概率为:

概率　排列

$$p(k) = 1 - P_{365}^{k}/365^{k}$$

$$k = 23, p(k) = 0.5073$$

$$k = 100, p(k) = 0.9999997$$

3.3 安全的哈希值-安全问题的思考

目前安全的哈希值位数是多少?

假设哈希值位数为L，其信息集合大小为 2^L，那么，这集合中多少个消息时使得出现碰撞的概率大于50%。

3.3 哈希值的安全性-哈希函数的攻击过程

Alice要支付的金额为1亿元
Alice要支付的金额为壹亿元
Alice应支付的金额为1亿元
Alice需要支付的金额为1亿元
……
还可以在交件中增加空格或文字颜色等等,这都不会影响合同的意思。

- ✓ 签名者对消息x的签名其实就是使用私钥对H(x)加密得到的签名值$Sig_k(h(x))$。
- ✓ 敌手对消息x产生2^k个变形的消息(即表达同样的意思消息)再拟定一个准备替换消息x的变形消息 x_1 (它表示另外一个意思，但对攻击者有利)，产生x_1的2^k个变形消息。计算所有这些消息的哈希函数值。
- ✓ 比较这两个哈希函数值集合，以便发现具有相同哈希值(匹配)的消息。如果没发现，则再产生其他一对有效消息和变形消息，直到出现一个匹配为止。
- ✓ 用找到匹配的变形消息x_1代替消息x，后面仍然附加签名值$Sig_k(h(x))$一起送给验证者。

3.3 哈希值的安全性-两[签名者认可的信息集合]相[攻击者希望的信息集合]

已知两个k元集合$X=\{x_1,x_2,\cdots,x_k\},Y=\{y_1,y_2,\cdots,y_k\}$，其中$x_i,y_j$($1\le i,j\le k$)是$\{1,2,\cdots,n\}$上均匀分布的随机变量。取定$x_i$，若$y_i=x_i$，则称$y_i$与$x_i$匹配。固定$i,j$，$y_i$与$x_i$匹配的概率为$1/n$，$y_i\neq x_i$的概率为$1-1/n$。$Y$中所有$k$个随机变量都不等于$x_i$的概率为$(1-1/n)^k$。假设$X,Y$中的$k$个随机变量两两互不相同，则$X$与$Y$中不存在任何匹配的概率为$(1-1/n)^{k^2}$。从而$Y$与$X$至少有一个匹配的概率为$1-(1-1/n)^{k^2}$。我们希望知道多大的$k$能使这个概率大于0.5。

为此，利用不等式$(1-x)\le e^{-x},x\ge 0$，有$1-(1-1/n)^{k^2}>1-(e^{-1/n})^{k^2}$。

令不等式右边等于0.5，求出$k=\sqrt{(\ln 2)n}\approx 0.83\sqrt{n}\approx\sqrt{n}$。

结论：n个元素的集合中，选取大约$\sqrt{n}$个元素时出现相同元素的可能性大于50%.

3.3 哈希值的安全性-结论

假设哈希函数H输出数据长度为L，全部可能的输出有$n=2^L$个。哈希函数H接受k个随机输入产生集合X，接受另外k个随机输出产生集合Y。根据前面的讨论知，当$k=\sqrt{n}=2^{L/2}$时，X与Y至少存在一对匹配(即哈希函数产生碰撞)的概率大于0.5。由此看出，$2^{L/2}$将决定输出长度为L的哈希函数h强碰撞的强度。

Contents 内容



1 概述

2 HASH算法

3 HASH安全性

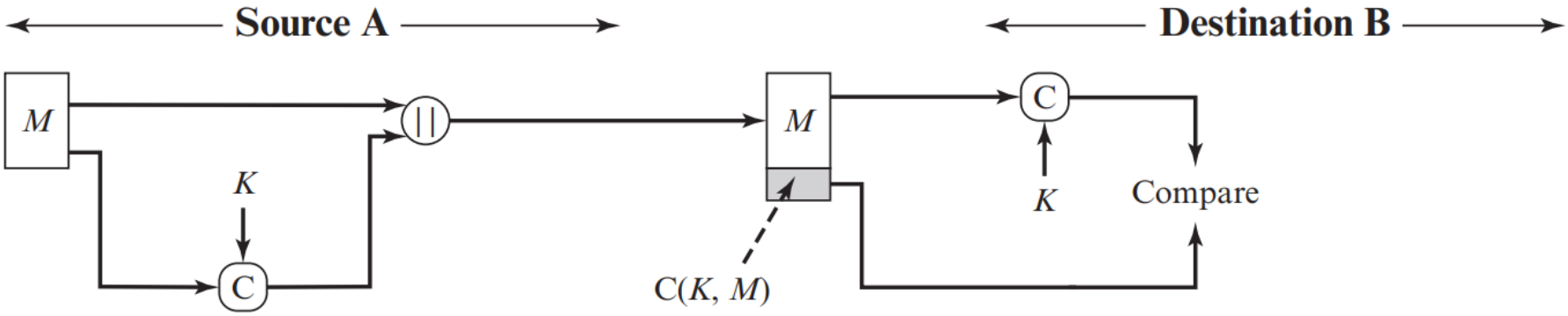
4 消息认证码

4.1 消息认证及消息认证码

- **消息认证**(message authentication)：验证所收到的消息确实是**来自真正的发送方**(source authentication)，而且消息**未被篡改过**(integrity)。
  - 通过为消息产生消息认证码(message authentication code)实现
  - 不解决发方**“不可否认”**
- **消息认证码（MAC）**：消息被一**密钥控制**的**公开函数**作用后产生的、用作**认证符**(authenticator)的、固定长度的数值，也称为密码校验和(cryptographic checksum)。
  - 要求通信双方共享有对称密钥K(shared symmetric secret key)
  - MAC函数（也称为keyed hash function）与加密算法类似，不同之处为MAC函数不要求是可逆的

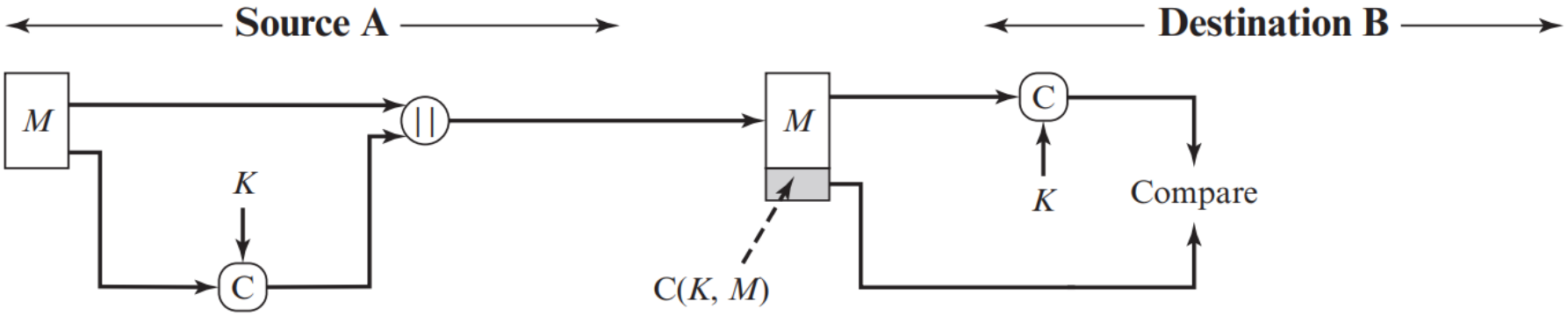
4.1 消息认证码的使用

- 设消息为M，C表示MAC函数，则$MAC = C(K, M)$
- 假定通信双方A和B共享密钥K, A发送给B的的消息是M：
  - A首先计算$MAC = C(K, M)$，然后向B发送$M||MAC$，B收到后做与A相同的计算，求得一新MAC，并与收到的MAC做比较。



4.1 消息认证码的使用

- 若验证通过，则说明：
  - **消息未被篡改**：这是因为攻击者不知道密钥，所以不能够在篡改消息后相应地篡改MAC，而如果仅篡改消息，则接收方计算的新MAC将与收到的MAC不同。
  - **接收方可以相信消息来自真正的发送方**。因为除收发双方外再无其他人知道密钥，因此其他人不能产生具有正确MAC的消息。



4.2 消息认证码的要求

- 消息认证码的穷举攻击的代价比使用相同长度密钥的加密算法的穷举攻击还要大。
  - 如果加密密钥长为k比特，则穷搜索攻击平均将进行2^{k-1}个测试。特别地，对惟密文攻击来说，敌手如果只知道密文C，则将对所有可能的2^k个密钥值执行解密运算，直到得到有意义的明文。

- 设MAC函数的密钥K长度为k比特（比如80），输出的MAC值为n比特（比如32），$k > n$

4.2 消息认证码的要求

- 若敌手已知M和$MAC = C(k, M)$，不知道K，则构建满足$C(k, M') = C(k, M)$的消息M'在计算上是不可行的。
  - 否则攻击者可以对消息进行篡改而不被发现。
- $C(k, M)$应该是均匀分布的，则对任意随机选择的消息M和M', $C(k, M') = C(k, M)$的概率是2^{-n},n为MAC值的位数。
  - 为了抵抗攻击者没有密钥但能访问MAC函数产生所选择的消息的MAC值的情况
- 设M'是M的某个已知变换，即$M' = f(M)$,例如f可能将M的一位或者多位取反，此时要求$C(k, M') = C(k, M)$的概率是2^{-n}。
  - MAC函数不应在消息的某个部分或某些比特弱于其它部分或其它比特，否则敌手获得M和MAC值后就有可能修改M中弱的部分，从而伪造出一个与原MAC相匹配的新消息。

4.2 对消息认证码的穷举攻击

- 给定若干M_i，$T_i = C(K, M_i)$，攻击者需要重复下述攻击以找到密钥:

第一轮：

- 给定$M_1, T_1 = C(K, M_1)$，对所有2^k个密钥，计算$T_i = C(K_i, M_1)$
- 匹配数约为2^{k-n}

第二轮：

- 给定$M_2, T_2 = C(K, M_2)$，对余下的2^{k-n}个密钥，计算$T_j = C(K_j, M_2)$
- 匹配数约为2^{k-2n}

以此类推。

- 考虑$k > n$。所有可能的2^k个密钥会产生2^k个MAC，但实际上只有 $2^n < 2^k$ 个不同的MAC值，即许多密钥会产生正确的MAC，但是攻击者不知道哪一个是正确的。
- 平均来说，有$\frac{2^k}{2^n} = 2^{k-n}$个密钥会产生正确的MAC。
- 因此需要进行一个重复的过程进行攻击。

4.2 对消息认证码的穷举攻击

- 给定若干$M_i, T_i = C(K, M_i)$，攻击者需要重复下述攻击以找到密钥:

第一轮：

- 给定$M_1, T_1 = C(K, M_1)$，对所有2^k个密钥，计算$T_i = C(K_i, M_1)$
- 匹配数约为2^{k-n}

第二轮：

- 给定$M_2, T_2 = C(K, M_2)$，对余下的2^{k-n}个密钥，计算$T_j = C(K_j, M_2)$
- 匹配数约为2^{k-2n}

以此类推。



- 平均来说，若$k = \alpha \times n$，则需α轮。
- 例：设$k = 80, n = 32$。第一轮会得到约2^{48}个密钥；第二轮会得到约2^{16}个密钥；第三轮则得等最终确定的密钥。

4.2 对消息认证码的穷举攻击

- 给定若干$M_i, T_i = C(K, M_i)$，攻击者需要重复下述攻击以找到密钥:

第一轮：

- 给定$M_1, T_1 = C(K, M_1)$，对所有2^k个密钥，计算$T_i = C(K_i, M_1)$
- 匹配数约为2^{k-n}

第二轮：

- 给定$M_2, T_2 = C(K, M_2)$，对余下的2^{k-n}个密钥，计算$T_j = C(K_j, M_2)$
- 匹配数约为2^{k-2n}

以此类推。



- 若密钥长度k小于或等于n，则很可能第一次循环就得到一个密钥，当然也可能得到多个，只要得不到唯一的密钥，攻击者就需要新的<消息，MAC>对进行测试。
- 因此，用穷举法来确定认证密钥不是一件容易的事。确定认证密钥比确定同样长度的加密密钥更困难。

4.3 基于哈希函数的MAC

- 基本思路

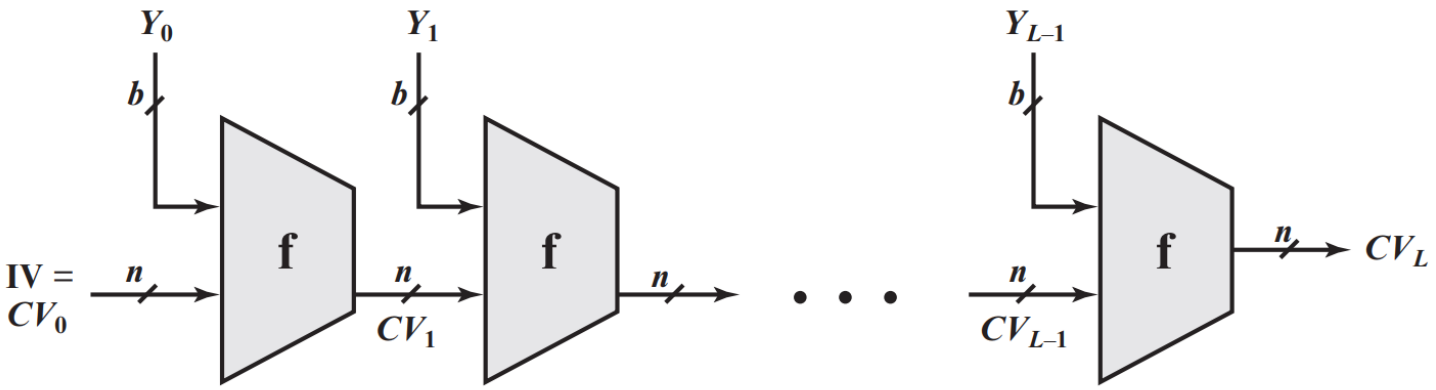
  将密钥和消息一起作为哈希函数的输入求哈希值

- 设计目标
  - 可以使用任何现有的哈希函数而需要对其进行修改
  - 一旦有更快或者更安全的哈希函数，能够容易的对原来的嵌入哈希函数进行替代
  - 能保持原有哈希函数的性能，不造成较大的性能下降
  - 对密钥的使用和管理要简单
  - 若已知嵌入的哈希函数的强度，则可以知道认证机制抗密码分析的强度

4.3 两种简单但不安全的构造

1. $T = \mathrm{MAC}_k(M) = H(k||M)$，称为秘密前缀(secret prefix) MAC
2. $T = \mathrm{MAC}_k(M) = H(M||k)$，称为秘密后缀(secret suffix) MAC

4.3 两种简单但不安全的构造-哈希函数结构



IV = Initial value
CV_i = Chaining variable
Y_i = ith input block
f = Compression algorithm

L = Number of input blocks
n = Length of hash code
b = Length of input block

- 将输入的消息划分为分组，对每个分组迭代的应用函数压缩f。

4.3 对秘密前缀MAC $h(k||M)$的攻击

- 设消息M为一个分组序列(sequence of blocks)，即$M = (x_1, \cdots, x_n)$
- 则$T = \mathrm{MAC}_k(M) = h(k||x_1, \cdots, x_n)$
- 此种构造方式的问题在于消息$M' = (x_1, \cdots, x_n, x_{n+1})$（其中$x_{n+1}$为任意的敌手设定的分组）的MAC值可以直接由$T$计算得出而不需要知道密钥$k$

4.3 对秘密前缀MAC $h(k||M)$的攻击

Alice **敌手** **Bob**

$M = (x_1, \cdots, x_n)$

$T = h(k||x_1, \cdots, x_n)$

截获(intercept)

M, T ←

$M' = (x_1, \cdots, x_n, x_{n+1})$

$T' = h(T||x_{n+1})$

M', T' ←

- 计算$h(k||M') = h(k||x_1, \cdots, x_n, x_{n+1})$
- 验证$T = h(k||M')$通过

- 该攻击可能成功是因为哈希函数的迭代结构。注意哈希函数中的压缩函数的输入为当前的分组和处理完前面分组得出的结果。
- 因此，添加一个新分组x_{n+1}之后，新的消息哈希值可以由前面计算得到的哈希值T作为压缩函数的输入得到。

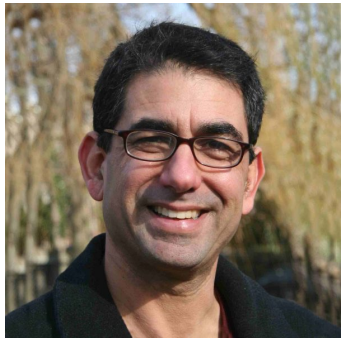
4.3 对秘密后缀MAC $H(M||k)$的攻击

- 假设敌手可以找到两个不同的消息M和M'，使得它们的哈希值相同 $h(M) = h(M')$
  - 由于哈希函数的迭代计算结构，$h(M||k) = h(M'||k)$
  - 即由一个消息M（或M'）的MAC值，即可得到另外一个消息M'（或M）的MAC值

- 该攻击的成功取决于哈希函数的具体参数（决定其抗碰撞性），**而不取决于密钥**。对于输出长度为160比特的哈希函数来说（比如SHA-1），**即使采用的密钥有128比特**，那么攻击的复杂度也只是**约2^{80}**。

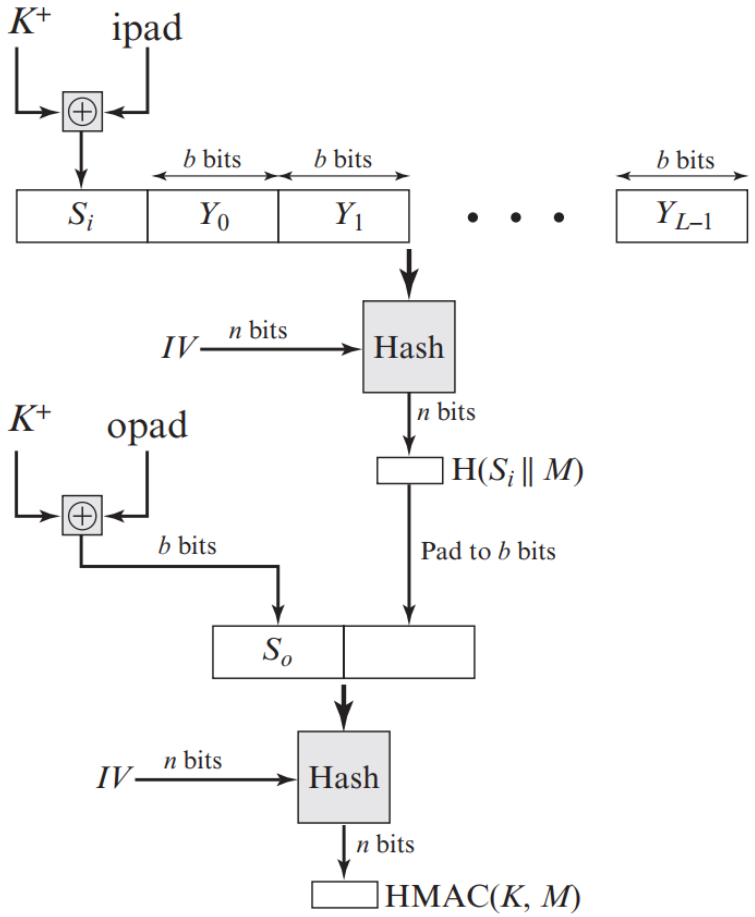
4.4 HMAC

- HMAC的构造由Mihir Bellare, Ran Canetti and Hugo Krawczyk于1996年提出
  - 避免了前面两种简单构造存在的安全问题。
  - 安全性具有严格的证明。


4.4 HMAC

- HMAC的构造为：$\mathrm{HMAC}_K(M) = H[(K^+ \oplus \mathrm{opad}) || H[(K^+ \oplus \mathrm{ipad}) || M]]$
  - K^+为K左边填充0得到，具有b位。（例如，若K的长度位160位，$b = 512$，则填充44个字节的0，即352位0）
  - b为哈希函数H输入消息的分组长度
  - H可以为任意现有的哈希函数
  - opad, ipad是常数
    - opad为01011100重复b/8次的结果
    - ipad为00110110重复b/8次的结果



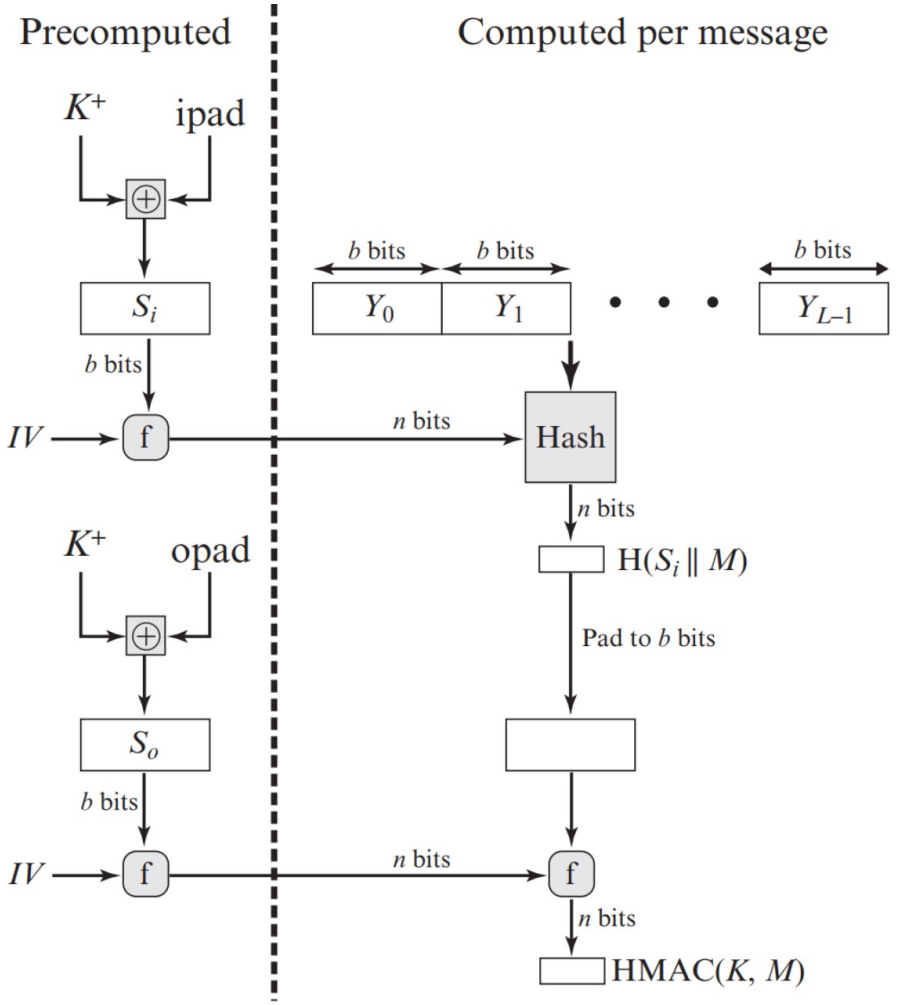
4.4 一种更高效的HMAC应用方式

- 在应用HMAC时，可预先求出下面两个值：

$$f(IV, (K^+ \oplus ipad))$$

$$f(IV, (K^+ \oplus opad))$$

- 这样一来，每当为一个消息计算HMAC值时，可以省去上面两个值的计算
  - 若无预先计算存储，则每次均要对这两个值进行计算



4.4 HMAC

- 文献证明成功攻击HMAC的概率等效于针对嵌入哈希函数的攻击
  - 比如生日攻击，敌手需要找到具有相同哈希值的两条消息M和M'。前面介绍过，该攻击的复杂度为$2^{n/2}$。
  - 像MD5这种输出哈希值只有128比特的哈希函数本身是不能保证安全性的。
  - 然而，HMAC本身含有密钥K,由于敌手不知道密钥，当哈希函数为MD5时，进行生日攻击至少需要**在线**连续观察同一HMAC密钥下发送的2^{64}个消息分组(512比特)的MAC值。2^{64}个消息分组(512比特)为2^{70}字节，在1 Gbit的网络连接下，需要250,000年。因此，MD5也可以用于HMAC中（不考虑其他可以降低碰撞攻击复杂度的机制的情况下）。

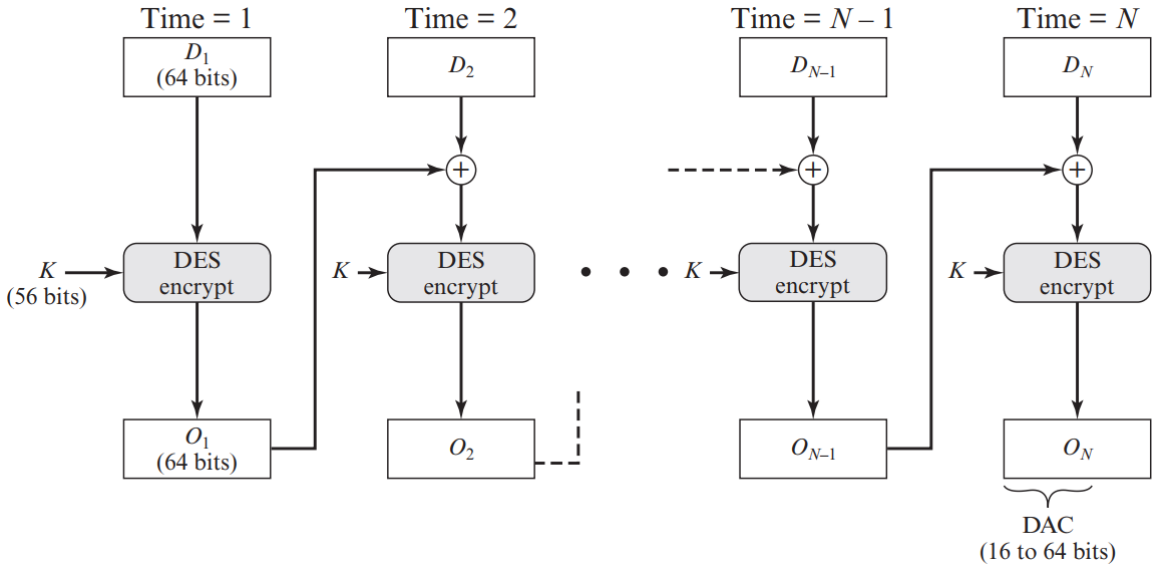
4.5 基于分组密码的MAC

- **数据认证算法**(Data Authentication Algorithm (DAA))
  - 该算法较陈旧，目前已被废止

- **基于密码的消息认证码**(Cipher-Based MAC (CMAC))

4.5 数据认证算法DAA

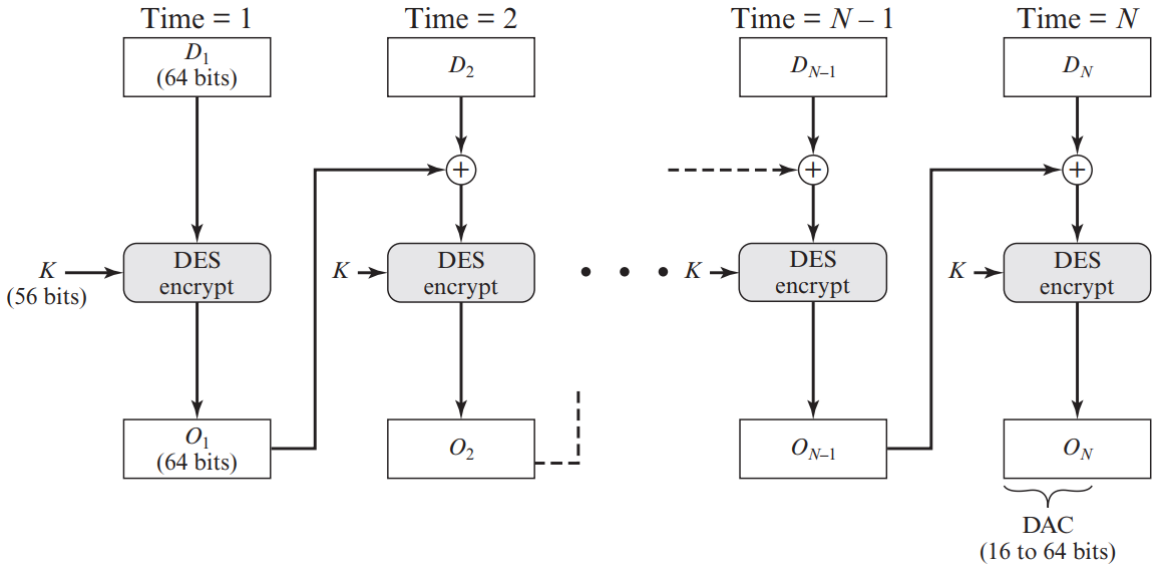
- DAA采用DES运算的密文分组链接(cipher block chaining (CBC))模式
- 其初始向量IV为0，需要认证的数据被分成连续的64位分组$D_1,\ldots,D_N$，最后一个分组不足64位时，在其后填充0，使其成为64位分组。



4.5 数据认证算法DAA

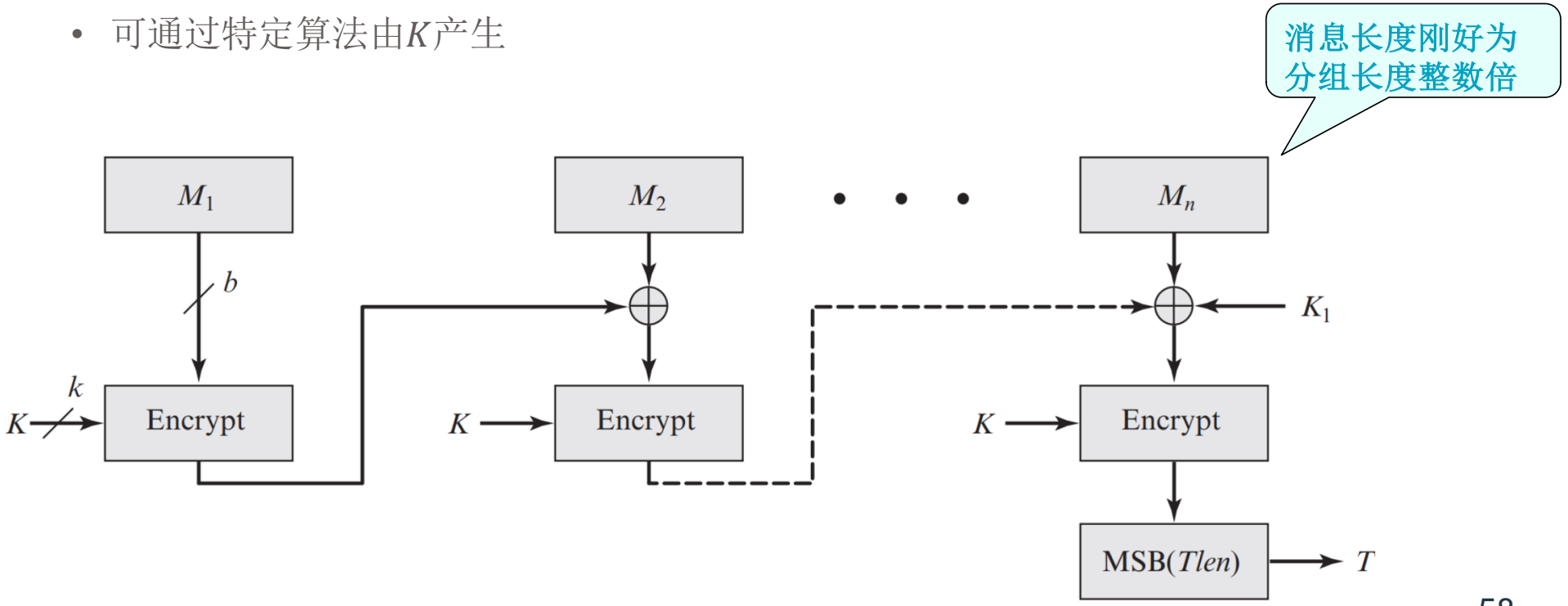
- 局限性
  仅能处理长度固定为mn的消息，其中n是密文分组的长度，m是一个正整数。

- 例如，已知单个分组长度的消息X,如果$T = MAC(K, X)$，则敌手马上就知道两个分组长度的消息$X \| (X \oplus T)$的CBC MAC码，因为它仍然是T。



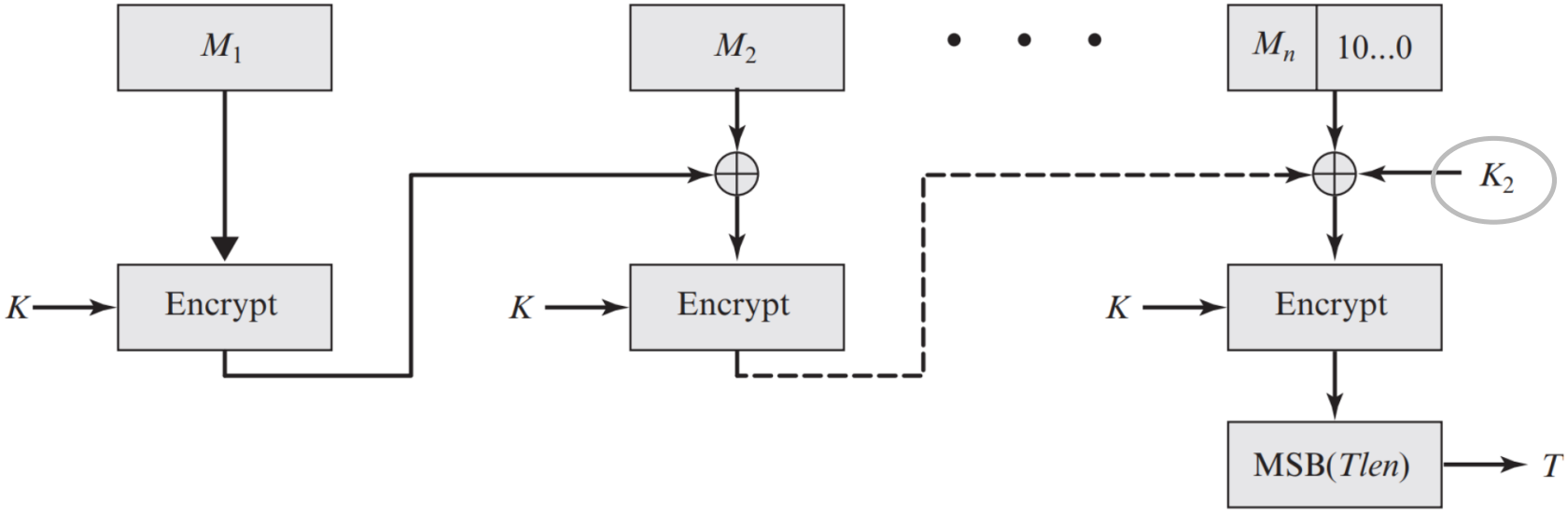
4.5 基于密码的消息认证码CMAC

- 为了克服DAA的局限性，使用三个密钥：
  - 一个长度为k位的分组密码算法加密密钥K,用在密文分组链接的每一步
  - 两个长度为b的密钥K_1, K_2
    - 可通过特定算法由K产生



4.5 基于密码的消息认证码CMAC

- 为了克服DAA的局限性，使用三个密钥：
  - 一个长度为k位的分组密码算法加密密钥K,用在密文分组链接的每一步
  - 两个长度为b的密钥K_1, K_2
    - 可通过特定算法由K产生



小结

- 哈希函数的简介
- 哈希函数算法举例(SHA256、SM3)
- 哈希函数的安全性
- 消息认证码

课后



1、证明：若HASH（CV_0，M_1） = X，则有HASH（CV_0，M_1 ｜ ｜M_2）=HASH（X，M_2）

2、哈希函数可用于哪些场景，从流程、关键数据结构等角度，展开描述其中一个应用

3、列举哈希函数与密码算法安全性的异同



谢 谢